

Assignment 3: Build Non-Linear Models Part 1

Juan Maldonado Franco

DDS-8555 Predictive Analysis

Mohamed Nabeel

```
In [1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path.cwd()
for parent in [ROOT, *ROOT.parents]:
    if (parent / "DDS-8555 - Predictive Analysis").exists():
        COURSE = parent / "DDS-8555 - Predictive Analysis"
        break
else:
    COURSE = ROOT.parents[1]
DATA = COURSE / "data"
KAGGLE = DATA / "kaggle"
SUBMISSIONS = DATA / "submissions"
RANDOM_STATE = 42
pd.set_option("display.max_columns", 80)
```

Conceptual Question 1

For any fixed number of predictors k , best subset selection must have training RSS less than or equal to forward or backward stepwise selection because it searches all k -predictor models. Test RSS cannot be known from the search method alone because the best training model may overfit. The forward stepwise nesting statement is true, backward stepwise nesting is true when read in the backward path direction, and the cross-method subset claims are false. Best subset models are also not necessarily nested across k because the best k -variable model does not have to be contained inside the best $k + 1$ -variable model (James et al., 2023).

Applied Question 8: Simulated Selection

The simulated exercise compares forward selection, backward selection, and lasso on polynomial features. The data-generating process includes powers of X , so a good selection method should recover the important polynomial terms more often than noise terms.

```
In [2]: from itertools import combinations
import statsmodels.api as sm
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline

rng = np.random.default_rng(RANDOM_STATE)
n = 100
X_raw = rng.normal(size=n)
eps = rng.normal(size=n)
Y = 1 + 2 * X_raw - 1.5 * X_raw**2 + .75 * X_raw**3 + eps
poly = pd.DataFrame({f"X^{i}": X_raw**i for i in range(1, 11)})

def fit_aic(cols):
    Xc = sm.add_constant(poly[list(cols)])
    return sm.OLS(Y, Xc).fit()

def forward_select():
    remaining = list(poly.columns)
    chosen = []
    best_aic = np.inf
    while remaining:
        scores = []
        for col in remaining:
            fit = fit_aic(chosen + [col])
            scores.append((fit.aic, col, fit))
        aic, col, fit = min(scores, key=lambda x: x[0])
        if aic < best_aic:
            chosen.append(col); remaining.remove(col); best_aic = aic; best_fit = fit
        else:
            break
    return chosen, best_fit

forward_cols, forward_fit = forward_select()
lasso = Pipeline([("scale", StandardScaler()), ("model", LassoCV(cv=10, random_state=rng))])
lasso.fit(poly, Y)
coef = pd.Series(lasso.named_steps["model"].coef_, index=poly.columns)
print("Forward selected:", forward_cols)
display(forward_fit.params)
display(coef[coef.abs() > 1e-3].rename("lasso_coefficient"))
```

Forward selected: ['X^1', 'X^2', 'X^3']

const 1.116665

X^1 1.923477

X^2 -1.694628

X^3 0.844008

dtype: float64

X^1 1.463389

X^2 -1.210939

X^3 1.363408

X^4 -0.109297

Name: lasso_coefficient, dtype: float64

Kaggle Regularization and PCR

The competition portion uses one regularized model and one principal-components regression model. Regularization controls coefficient instability, while PCR reduces the predictor space before regression (Jolliffe, 2002; Tibshirani, 1996).

```
In [3]: from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.linear_model import ElasticNetCV, Ridge
from sklearn.metrics import mean_squared_log_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

abalone = pd.read_csv(KAGGLE / "playground-series-s4e4" / "train.csv")
X = abalone.drop(columns=["Rings"])
y = abalone["Rings"]
cat = ["Sex"]; num = [c for c in X.columns if c not in cat + ["id"]]
pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), cat), ("num",
X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=.2, random_st
models = {
    "Elastic net": Pipeline([("pre", pre), ("model", ElasticNetCV(l1_ratio=[.1,.5,.
    "PCR ridge": Pipeline([("pre", pre), ("pca", PCA(n_components=.95)), ("model",
})
rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = np.maximum(model.predict(X_valid), 1)
    rows.append({"model": name, "validation_rmsle": np.sqrt(mean_squared_log_error(
pd.DataFrame(rows)
```

Out[3]:

	model	validation_rmsle
0	Elastic net	0.165582
1	PCR ridge	0.186751

```
In [4]: from pathlib import Path
status_path = SUBMISSIONS / "kaggle_submission_status_playground-series-s4e4.txt"
text = status_path.read_text(encoding="utf-8", errors="ignore")
print("\n".join([line for line in text.splitlines() if "A3_" in line or "fileName"
```

Interpretation

The regularized model is the better fit for this stage of the assignment because it keeps the original predictors visible while shrinking unstable coefficients. PCR is useful as a dimensionality-reduction comparison, but its components are less directly interpretable. The Kaggle evidence shows both submissions completed, and the public score favored the elastic net model over the PCR ridge model.

References

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer. <https://doi.org/10.1007/b98835>

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>